TraceClaw: Source-Anchored Auditing of the LLM Agent Control Plane

Anonymous Author(s)

October 2026

Abstract

Most agent evaluations focus on the final answer. TraceClaw studies a different object: the control plane that authenticates a connection, authorizes a method, resolves Session and Agent state, admits work, and hands the request to the deeper runtime. We present a source-anchored ledger for auditing this path in OpenClaw v2026.7.1-2 (0790d9f593ad 30c940ed93b5872a8cf6d6f3cf8c) with a TraceClaw instrumentation patch applied. Each Gateway stage is written as a falsifiable claim with source anchors, case values, observation status, and a branch outcome. In a seven-run controlled cohort, all runs cover the same G0–G18 Gateway path (cov = 19/19) and directly emit the stage outcomes (obs = 19/19). Source anchoring turns these successful agent executions into checkable evidence: R1/R7 and R5/R6 repeat the same prompts with identical Gateway branches, R2 exercises read-only retrieval, R4 exercises a local workspace mutation, and R3 exposes an external side-effect path whose command completed but whose downstream delivery is not verified. We also separate stage-outcome observation from rationale visibility: even when a stage outcome is observed, internal predicates and returned fields may remain only partially visible.

Keywords: LLM agents, control plane, runtime auditing, source anchoring, provenance, OpenClaw

1 Introduction

An LLM agent request involves more than a single model invocation. Before a model answer is produced, the surrounding system has already made a sequence of control-plane decisions: whether the connection is authenticated, whether the method is authorized, which Session and Agent own the request, whether the run is new, whether work is admitted, and which reply resolver receives control. These decisions affect security and reliability, but they are often hidden behind the final natural-language response.

Against this background, a satisfactory answer does not necessarily make the path that produced it easy to audit. An auditor may still need to determine whether an accepted request used a shared token or a device-token fallback, whether an administrator shortcut was taken, whether duplicate-run protection admitted or suppressed the work, or whether a tool-side effect was supported by an explicit authorization event or executed inside a shell command. These questions concern the agent control plane rather than answer quality. The final response, and even a high-level architecture diagram, usually does not provide enough evidence to answer them.

To make these decisions auditable, TraceClaw treats the control plane as the audit object. We represent each stage of a concrete request as a small, checkable claim: which source range defines the decision, which runtime value selects the branch, which runtime event supports the outcome, and which nearby branches were not taken. Ordinary tracing shows what executed, while provenance records where data or artifacts came from. TraceClaw connects these forms of evidence to the branch decisions of a particular agent request and records whether the rationale is directly observed or derived from source.

This connection matters at the decision boundary. A trace event may show that a stage returned `allow` without revealing which predicate made that branch valid. Source code can show the possible branches, but without a saved run it cannot establish which one the request actually took. TraceClaw therefore links the source decision site, the concrete run value, the resulting branch, and the visibility of the rationale.

In our controlled evaluation, we study OpenClaw v2026.7.1-2 with a TraceClaw instrumentation patch. R1 is the primary controlled case, using the prompt *How to make a cake?*, and R7 repeats the same prompt under the frozen environment. R5 and R6 form a second exact-prompt pair for a tracing-versus-provenance explanation. R2, R3, and R4 exercise read-only retrieval, an external side-effect path, and a local workspace mutation, respectively. Section 8 discusses the provenance strength of these runs and the metadata that remains unavailable.

2 Research Questions and Contributions

We ask three research questions.

1. **RQ1: Gateway Path.** Can a concrete request be followed through all G0–G18 Gateway stages without relying on architecture diagrams alone?
2. **RQ2: Evidence Quality.** For each stage, what is directly observed, what is source-derived, and what internal rationale remains only partially visible?
3. **RQ3: Runtime Return.** After G18 selects the reply resolver, can the Agent Runtime lifecycle and the resolver return edge be linked back to Gateway post-processing?

Our contributions are stated as claims supported by concrete evidence.

1. **Control-plane audit framing.** We treat authentication, authorization, Session/Agent routing, duplicate-run checks, work admission, resolver selection, and resolver return as explicit audit targets.
2. **Source-anchored ledger.** We define a stage ledger $E_i = (C_i, V_i, O_i, B_i)$, covering source anchors, concrete values, observation status, and branch outcomes. Across R1–R7, all 19 Gateway stages are covered and directly observed at the stage-outcome level.
3. **Rationale-gap accounting.** We separate stage-outcome observation from rationale visibility. The controlled traces expose all stage outcomes, while rationale visibility remains partial because internal predicates, stdout from short probe commands, and some return fields are not fully emitted.
4. **Runtime-return analysis.** We apply the same ledger idea to a separate AR0–AR6 runtime layer and show how no-tool, retrieval, local-mutation, and external side-effect executions differ.

3 Related Work

OpenClaw architecture. OpenClaw architecture documentation describes the broad control plane, Sessions, Gateway, event loop, and observability goals [11, 12, 13]. These materials are useful for understanding the overall structure of the system, but we do not use them as direct evidence for individual stage claims. Instead, we re-derive the G0–G18 stage labels and branch claims from the pinned OpenClaw source and saved traces.

Tracing and provenance. Dapper makes large-scale distributed execution visible through spans and trace context [1], while OpenTelemetry provides a modern tracing framework for

recording and propagating execution information [2]. W3C PROV-DM provides a data model for describing the provenance of entities, activities, and their relationships [3]. These systems address questions related to ours, but our focus is narrower: we ask which concrete value selected a control-plane branch in a particular run, and which alternative branch was therefore excluded.

This distinction matters in agent systems because many control-plane decisions are not naturally represented as spans. A span may show that authorization for `chat.send` completed, while we may still need to determine which scope made the call legal and whether a more privileged shortcut was skipped. A provenance relation may show that a reply depends on a tool result, but it may not explain the policy gate, duplicate-run check, or resolver decision that led to the tool execution. We therefore treat traces and provenance records as evidence sources rather than complete explanations.

Dynamic program analysis. Weiser's program slicing identifies statements that may influence a program value [4], while Agrawal and Horgan's dynamic slicing ties that analysis to a concrete execution [5]. Daikon takes a different approach by inferring likely invariants from observed executions [6]. Our design follows the same execution-specific perspective, but the output is different: we produce an audit ledger for a concrete agent request rather than a program slice or an inferred invariant set.

In our design, the ledger is intentionally less automatic than slicing. We do not attempt to compute every statement that may have influenced a value. Instead, we record the source range, concrete runtime value, observed outcome, and nearby branches that did not fire. When a relevant value is absent from the saved trace, we keep the conservative label `NOT RECORDED` rather than infer a concrete value from source code alone. This keeps the evidence inspectable while preserving the distinction between observed and source-derived claims.

Agent debugging and runtime observability. XAgen studies explainability and failure analysis in multi-agent workflows [7], while FlowForge and visual analytics for coding agents use visualization to expose agent workflow structure and behavior [9, 8]. Fangcun Observer focuses more directly on runtime security signals such as commands, file activity, and behavior chains [10]. Our approach starts from a different boundary: we begin with Gateway control-plane decisions and connect them to later runtime evidence.

The closest connection appears in the post-G18 runtime layer. After the Gateway selects a resolver, we follow the deeper execution and record whether the run is no-tool, retrieval-only, a local workspace mutation, or an external side-effect execution. We then link the resolver-return event back to the Gateway source path. This provides a repeatable way to separate control-plane evidence, runtime evidence, and source-derived return-path claims.

4 Method: Source-Anchored Runtime Trace

4.1 Audited System Boundary

TraceClaw is a source-anchored auditing method for agent control planes. We use it when a concrete execution claim needs to be checked against both the implementation and the saved runtime evidence. The method has four steps.

1. **Choose an audited request family.** We keep one primary request for the detailed source audit and add comparison or repeated runs when saved traces are available. We preserve the request values that may affect control flow, including the Session key, run ID, message, Agent ID, and authorization scope.

2. **Reconstruct the source path.** We follow the request from connection authentication through authorization, routing, work admission, and reply-resolver selection, then record the handoff to the Agent Runtime.

3. **Attach runtime evidence.** For each stage, we align the relevant source decision with concrete runtime values and events. When an outcome is inferred from source and earlier observed values rather than emitted directly, we mark it as source-derived.

4. **Separate observed outcome from rationale visibility.** A stage may have an observed outcome even when some internal predicates, intermediate fields, or return values are not emitted. We keep these gaps explicit rather than treating them as observed.

Figure 1 summarizes the execution boundary we audit in OpenClaw v2026.7.1-2. A `chat.send` request first crosses the Gateway control plane, where authentication, authorization, request validation, Session and Agent routing, send policy, duplicate-run checking, work admission, context construction, and resolver selection take place. At G18, the selected resolver hands execution to the Agent Runtime.

Within the runtime, tool use is optional. AR2 records the tool/no-tool outcome for the run, while individual `tool_started` and `tool_result` pairs are tracked separately when tools are used. The final reply is then finalized, the Agent run ends, and the resolver returns. We mark the subsequent return/resume into Gateway post-processing as source-derived because that later Gateway step is not emitted as a separate runtime event.

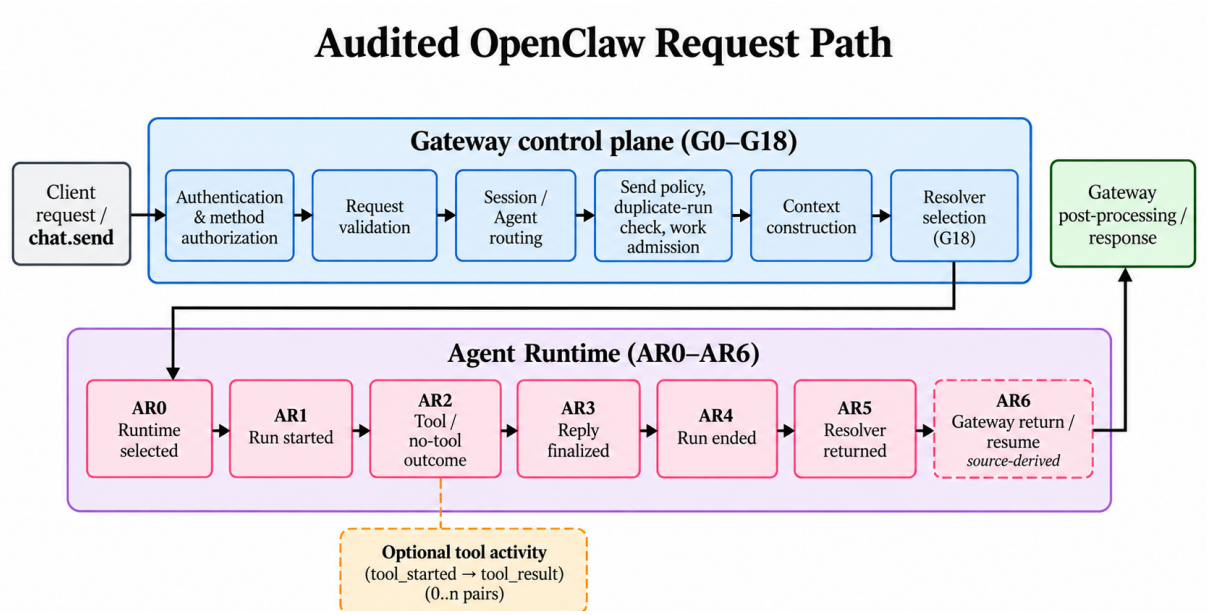


Figure 1: Audited OpenClaw request path. The G0–G18 Gateway control plane hands the selected resolver to the AR0–AR6 Agent Runtime. Tool activity is optional at AR2. AR5 records the observed resolver-return event, while AR6 denotes the source-derived return/resume link into Gateway post-processing.

We treat the Gateway and Agent Runtime as linked but distinct audit layers. G0–G18 captures the control-plane decisions that lead to resolver selection, while AR0–AR6 captures the deeper execution and its return to Gateway processing. This separation lets us compare Gateway branch evidence with runtime behavior without mixing the two stage sets.

4.2 Trace Model and Evidence Ledger

The main design choice is claim-level tracing. Instead of describing a run only as a narrative, we represent each audited stage as a small checkable claim. Each stage asks four questions: which source location defines the decision, which concrete value applies to this run, what evidence supports the outcome, and which nearby branch was not taken.

For a request r, the Gateway path is

$$\tau_G(r) = \langle s_0, \ldots, s_{18} \rangle.$$

Each Gateway stage s_i has an evidence item

$$E_i = (C_i, V_i, O_i, B_i),$$

where C_i is the set of verified source anchors, V_i contains concrete values from the run, O_i is the observation status, and B_i is the branch or stage outcome.

A Gateway stage is covered when it has at least one verified source anchor and at least one concrete run value. We therefore define

$$\mathrm{cov}(\tau_G) = \frac{1}{19} \sum_{i=0}^{18} \mathbf{1}\left[C_i \neq \varnothing \ \wedge \ V_i \neq \varnothing\right].$$

Direct stage-outcome observation is measured separately:

$$\mathrm{obs}(\tau_G) = \frac{1}{19} \sum_{i=0}^{18} \mathbf{1}\left[O_i = \mathrm{Observed}\right].$$

Here, obs measures whether the outcome of a stage is directly observed. It does not mean that every predicate, intermediate value, stdout string, token, or returned field explaining that outcome is emitted.

We therefore record rationale visibility separately as FULL, PARTIAL, or SOURCE-DERIVED. In the controlled cohort, every run has Gateway cov = 19/19 and obs = 19/19, while rationale visibility remains PARTIAL across the 19 Gateway stages. The stage outcomes are observed, but some internal predicates and returned state remain unavailable.

The post-G18 Agent Runtime is modeled separately as AR0–AR6. AR2 is one logical tool/no-tool stage. For tool-using runs, individual `tool_started` and `tool_result` events are retained, and tool-call completeness is reported separately as

$$\frac{\#\,\texttt{tool_result}}{\#\,\texttt{tool_started}}.$$

For no-tool runs, AR2 records the observed no-tool outcome when the Agent lifecycle completes without tool events. AR6 is source-derived: the observed `reply_resolver_returned` event establishes return to the dispatch source point, while the later Gateway post-processing step is not emitted as a separate runtime event.

This distinction allows a run to have cov = 19/19 and obs = 19/19 while still containing field-level gaps. The appendix therefore keeps `NOT RECORDED` values explicit without reducing stage coverage when sufficient source and run evidence exists to audit the stage outcome.

In Figure 2, ρ denotes the return edge from the Agent Runtime to the Gateway dispatch path. The edge is source-derived rather than a separately emitted Gateway runtime event.

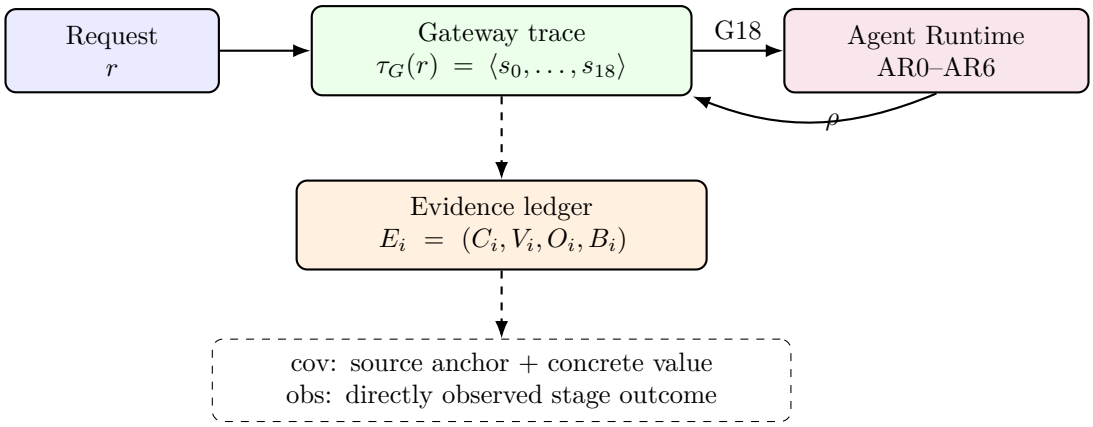


Figure 2: Mathematical trace model. A concrete request is mapped to an ordered Gateway stage sequence, a separate post-G18 Agent Runtime layer, and an evidence ledger. The return edge ρ links the runtime return back to Gateway processing.

Table 1: Positioning TraceClaw against common approaches.

Approach	**What it usually shows**	**What it misses**	**TraceClaw difference**
Architecture overview	Main components and high-level data flow.	Concrete branch choices, request values, and code-level evidence.	Each audited stage is tied back to source and case evidence.
Log-only analysis	Events emitted during one run.	Silent branches and the source reason for a decision.	Runtime events are aligned with source decisions and nearby branches.
Static code reading	Possible paths in the implementation.	Which path a concrete request actually took.	Source branches are bound to concrete run values and runtime evidence.
Model-output evaluation	Quality or safety of the final model response.	Gateway admission, routing, duplicate-run handling, and resolver return.	The audited object is the execution path rather than only the final answer.

5 Audit Workflow

The TraceClaw workflow combines three forms of evidence for each saved run: a pinned source snapshot, TraceClaw instrumentation, and the saved trace artifact. We use these inputs to reconstruct the Gateway and Agent Runtime paths, align observed values with source decisions, and record where the available evidence stops.

For each run, we first reconstruct the G0–G18 Gateway path and the post-G18 AR0–AR6 runtime path. We then align the observed events and concrete values with the corresponding source decisions. Finally, we record stage coverage, observation status, runtime class, and rationale-level gaps. Repeated prompts follow the same procedure so that their branch structures can be compared without changing the evidence model.

The workflow does not assume that every internal field will be available. Instead, it records the strongest claim supported by the source and runtime evidence. A directly observed stage outcome can therefore coexist with a partially visible rationale, and a runtime return can be source-derived even when the preceding resolver-return event is directly observed.

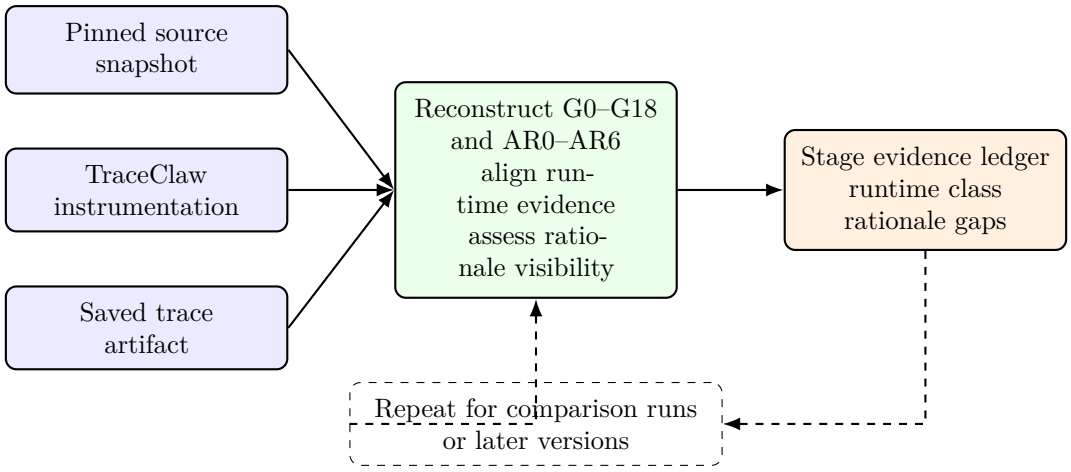


Figure 3: TraceClaw audit workflow for one run. The dashed box shows how the same procedure is repeated across prompts or versions.

This workflow also separates evidence collection from comparison. The same ledger structure is first applied to each individual run; only then are repeated prompts or different runtime classes compared. This avoids using agreement between runs as a substitute for evidence within a run.

6 Primary Controlled Case (R1)

R1 is the controlled replacement for the old Cake2 case. Its prompt is `How to make a cake?`. The old Cake2 artifact is kept only as historical context because it was collected under earlier instrumentation and used a different authentication path.

R1 is a baseline, not the main novelty by itself. The prompt is ordinary on purpose: the value of the case is that an apparently simple accepted request still crosses authentication, method authorization, Session resolution, Agent selection, send policy, duplicate-run checking, admission, context construction, resolver selection, and resolver return. If this baseline path cannot be audited carefully, then stronger claims about tool use or side effects would rest on a weak foundation.

R1 takes the shared-token path. G0 records token authentication material, G1 allows the shared credential, G2 finishes with role `operator` and scope `operator.write`, and G3 authorizes `chat.send` through the write check. The administrator shortcut is not an R1 claim. R1 records `hasDeviceIdentity=False`; the device-identity and device-signature branches are therefore not used for the R1 branch and are treated as source-derived not-taken paths. The source establishes only this rule: if `operator.admin` is present, authorization can short-circuit; otherwise `chat.send` requires `operator.write`. The old Cake2 trace and R1 differ on scope, but this paper does not claim that authentication method alone causes that scope.

The source-anchored ledger prevents a common mistake here. The trace directly shows the R1 branch outcome and the observed scopes. The source explains which branches those scopes make possible. The small cohort does not establish the causal path from authentication method to resulting scope, so we do not state that path as a finding.

From G4 onward, R1 follows the same successful Gateway branch structure as the later controlled runs: request validation passes, the message is unchanged, no explicit Agent override is supplied, the Session resolves, Agent `main` is selected, send policy allows the turn, the run is new, work is admitted, the message context is built, and the default `getReplyFromConfig` resolver is selected at G18. The supplementary appendix gives the detailed per-stage source audit for R1, including source anchors, concrete values, taken and not-taken branches, observation status, rationale visibility, and not-recorded fields.

The baseline also fixes what the later controlled runs are compared against. R2, R3, and R4 do not need a new Gateway theory; they reuse the same G0–G18 ledger shape while exercising different AR classes. R5/R6 and R1/R7 then test whether exact-prompt repeats preserve the same Gateway branch structure while changing run-specific identifiers.

7 Agent Runtime and Return Path

The AR layer complements the Gateway audit by answering a different question. G0–G18 records how a request reaches the selected resolver, while AR0–AR6 records what happens after that handoff and how execution returns to Gateway processing. This distinction lets us separate no-tool runs from read-only retrieval, local workspace mutation, and external side-effect execution without treating the final model answer as the only result of interest. As summarized in Figure 1, the seven AR stages cover runtime selection, run start, the tool/no-tool outcome, reply finalization, run completion, resolver return, and the source-derived resume into Gateway processing.

A no-tool run still provides positive evidence: the Agent lifecycle completes without tool events, so the absence of tool use is recorded rather than assumed. For tool-using runs, we retain the sequence of individual `tool_started` and `tool_result` events instead of reducing the execution to a single "used tools" flag. We also keep tool count, result count, stop reason, final reply, and resolver return separate because each supports a different claim.

Across R1–R7, every controlled run selects Agent `main`, uses the `embedded` runner, and runs with provider `openai` and model `gpt-5.5`. The terminal runtime sequence is also consistent across the cohort: `agent_reply_finalized`, followed by `agent_run_ended` with stop reason `stop`, and then `reply_resolver_returned`. Each run returns a single reply result.

AR6 captures the return edge ρ shown in Figure 2. We mark it as source-derived because the observed resolver-return event establishes that control has returned to the dispatch source point, while the later Gateway processing is not emitted as a separate runtime event. The stage labels identify audit decision sites rather than call-stack nesting: G18 is invoked within the G16 reply-dispatch path, so resolver return resumes G16 processing before the surrounding G14 orchestration returns.

The run-level runtime classes and tool evidence are summarized in Table 2.

8 Controlled Experiments

8.1 Cohort

Our controlled cohort contains seven saved runs, namely R1 through R7. Runs R1 through R5 are supported by saved trace artifacts and repository history. For these runs, our TraceClaw collection and instrumentation code remained stable, while the OpenClaw environment was supported by post-hoc source and instrumentation audit rather than by trace-embedded environment fingerprints. Runs R6 and R7 were collected at a later stage and contain direct before-and-after fingerprint evidence for the local OpenClaw checkout and instrumentation patch.

Table 2 summarizes these seven runs, their runtime classes, observed tool evidence, and provenance strength. Historical Weather traces and the older Cake2 artifact are retained solely as supplementary background information and are not included in the quantitative results for R1–R7; their details are reported in the supplementary appendix.

Table 2: Controlled cohort, runtime classes, tool evidence, and provenance. Tool ratios denote observed `tool_result`/`tool_started` events. All runs have Gateway coverage and observation of 19/19; AR0–AR5 are directly observed and AR6 is source-derived. R1–R5 use post-hoc environment audit, while R6/R7 additionally have direct fingerprints.

Run	Request	Runtime class	Tool evidence	Provenance
R1	cake baseline	no-tool	no tool events	post-hoc audit
R2	OpenTelemetry retrieval	read-only retrieval	`web_search` ×6, 6/6	post-hoc audit
R3	redacted email action	external side effect	`bash` ×5, 5/5	post-hoc audit
R4	Fibonacci file creation	local workspace mutation	`bash` ×3 + `apply_patch`, 4/4	post-hoc audit
R5	tracing/provenance explanation	no-tool	no tool events	post-hoc audit
R6	tracing/provenance repeat	no-tool	no tool events	fingerprinted
R7	cake repeat	no-tool	no tool events	fingerprinted

8.2 Gateway Coverage and Rationale Visibility

All seven of our controlled runs satisfied cov = 19/19 and obs = 19/19 at the Gateway layer. This means that each of the G0–G18 stages had at least one source anchor and one concrete run value, and that each stage outcome was directly observed. This does not mean that every internal predicate or returned field is visible. For all 19 Gateway stages, rationale visibility remains `partial`. Internal sub-branches, authorization rationale, and certain return payload fields are not fully emitted.

Table 3 lists representative rationale-level gaps. The complete stage-by-run matrix and the detailed R1 source audit are included in the supplementary appendix.

Table 3: Representative rationale-level gaps in the controlled cohort. Each audit point has an observed outcome, while some rationale remains unavailable in the saved evidence.

Audit point	Observed outcome	Rationale not fully emitted
G0/G2	shared-token authentication succeeds; the device-token path is not used	concrete device-token candidate and signature subpath
G3	`chat.send` is allowed through `operator.write`	authorization rationale and administrator-shortcut evaluation
G7/G8/G12	Session resolves, Agent/Session validation passes, and work is admitted	persistent-store details, full Session state, and admission internals
G10	send policy outcome is `allow`	exact internal allow sub-branch and policy rationale
G14/G16/G18	reply dispatch is entered, the normal path continues, and the resolver is selected	full dispatch/reply payloads and post-return Gateway state
AR/R3	five `bash` tool calls complete with exit code 0	probe stdout, separate approval event, and delivery confirmation

8.3 Runtime Cases

Here, we compare R2, R3, and R4 in more detail. These three tool-using runs represent different runtime classes while retaining the same successful G0–G18 Gateway structure. Specifically, R2 is a read-only retrieval run, R4 performs a local workspace mutation, and R3 executes an external side effect through a host application.

R2: Read-only retrieval. R2 searches for OpenTelemetry tracing documentation and summarizes it. The runtime records six serial `web_search` start/result pairs. Based on the recorded

tool-result metadata, the calls search OpenTelemetry tracing documentation, open the Signals page, search for trace concepts, open and revisit the Traces page, and finally open the Trace SDK page. The observed tool sequence lasts approximately 37 seconds before `agent_reply_finalized`, `agent_run_ended`, and `reply_resolver_returned` are recorded.

The Gateway path remains unchanged from the no-tool baseline; the difference appears after G18. The AR layer therefore distinguishes read-only retrieval from a no-tool response.

R4: Local Workspace Mutation. R4 requests a short Python script that prints the first 10 Fibonacci numbers. During execution, the runtime records a four-step local sequence: `pwd`, `rg --files`, an `apply_patch` operation that creates `fibonacci.py`, and `python3 fibonacci.py`. The observed tool sequence lasts approximately 10 seconds.

The evidence supports a local workspace mutation associated with this trace. The Gateway authorizes and dispatches the request, while the later file mutation is established by the AR tool sequence rather than by the Gateway ledger alone.

R3: External side effect. Compared with R2 and R4, R3 requests an operation to draft and send an email. The trace records four brief capability probes using `command -v ... || true` for `osascript`, `mutt`, `mail`, and `msmtp`. All four commands exit with status 0, but their standard output is not recorded. Because each probe ends with `|| true`, the exit status alone does not indicate whether the corresponding executable was actually found. The trace therefore does not directly reveal why `osascript` was selected.

The fifth command invokes Apple Mail through `osascript` and exits with status 0. The final assistant reply states that the email was sent, while the saved trace shows only that the host-side command completed; it does not verify downstream delivery. The reply therefore claims more than the saved trace can independently verify.

R3 also exposes a gap in approval and tool-policy observability. The trace directly records the `bash` execution, while the effective tool-policy path is source-derived from the G16 dispatch path. No separate approval or delivery-confirmation event is recorded.

8.4 Repeatability

R1/R7 and R5/R6 are exact-prompt repeat pairs. In both pairs, the G0–G18 branch structure, selected Agent, resolver, tool/no-tool outcome, stop reason, and AR structure match. Run-specific values, including the run ID, Gateway Session key, emitted AR identifiers, timestamps, and reply wording, change between executions as expected.

Both repeat pairs are no-tool runs. They test repeatability of the Gateway path and the no-tool Agent Runtime path, not repeatability of tool-using behavior. The repeat also provides a falsifiability check: if the same prompt under the controlled setup changed Gateway branches without a recorded reason, the stage model would be suspect. Within the two observed pairs, the branch structure remains stable. R7 serves as an exact-prompt repeat of the R1 baseline without being treated as a separate execution class.

8.5 Findings

The controlled experiments support three main findings. First, R3 shows why final-response claims and runtime evidence should be kept separate: the reply states that the email was sent, but the trace establishes only host-side command completion (Section 8.3). TraceClaw preserves the narrower auditable claim rather than treating the final response as proof of the external effect.

Second, the Gateway control-plane structure remains stable across the seven successful controlled runs even though post-G18 runtime behavior varies. The same G0–G18 path can lead to no-tool execution, read-only retrieval, local workspace mutation, or an external side effect. This supports analyzing the Gateway and AR layers separately.

Third, complete stage-outcome observation does not imply complete rationale visibility. All seven runs have Gateway cov = 19/19 and obs = 19/19, while all 19 Gateway stages retain `partial` rationale visibility. AR6 is source-derived rather than directly emitted as its own runtime event.

9 Limitations

Our evaluation is intentionally narrow. It covers seven successful requests collected on the same machine and under the same broad configuration. Each tool-using runtime class is represented by only one controlled run. The seven runs also share the same general authorization shape: shared-token authentication, role `operator`, and ordinary `operator.write` authorization for `chat.send`. As a result, the current study does not test how TraceClaw behaves under other authentication, authorization, or policy branches.

We also focus on successful executions. Rejected authentication, failed authorization, retries, cancellation, cross-version behavior, and multi-framework executions are outside the current evaluation. Some of these cases may terminate early or follow a different control path, so the present results should not be read as coverage of all OpenClaw behaviors.

Environment provenance is another limitation. The saved traces do not directly contain the OpenClaw commit, TraceClaw commit, trace-schema version, configuration fingerprint, or instrumentation patch hash. For R1–R5, we reconstruct this context from saved trace artifacts, repository history, and post-hoc source and instrumentation audit. R6/R7 have stronger provenance evidence because the local OpenClaw checkout and instrumentation patch were fingerprinted before and after collection.

Finally, parts of the method still require manual inspection. The G0–G18 stage labels are defined by TraceClaw rather than by OpenClaw itself, and source anchors must be checked against the relevant code. For this reason, the appendix cites the upstream 0790d9f decision sites instead of the instrumentation lines that emit runtime events. The event shows that something happened; the upstream source identifies the decision that the event is intended to support.

10 Artifact and Future Work

The accompanying artifact includes the trace viewer, saved controlled runs, generated evidence tables, and a sanitized reference to the instrumentation patch. For anonymous review, identifying information such as author URLs, email addresses, hostnames, usernames, and absolute local paths is removed or replaced with anonymous placeholders.

There are several clear ways to extend the current work. One is to make environment provenance part of the trace itself. Information such as the OpenClaw commit, TraceClaw commit, trace-schema version, configuration fingerprint, and instrumentation patch hash could be recorded when a run is collected. This would make later verification easier and reduce the amount of provenance that has to be reconstructed from repository history.

The current experiments could also be extended beyond successful runs. Useful cases include rejected authentication, failed authorization, duplicate-run suppression, cancelled execution, tool-policy denial, and cross-version replay. These paths are especially important because some of them may stop before reaching the full Gateway or Agent Runtime path. Testing them would show how well the same evidence model works when a run fails early or follows a different branch.

A further extension is to turn the current ledger into an interactive evidence graph. Each stage could be connected directly to its source anchors, runtime events, case values, and rationale-visibility status. This would make it easier to move from a high-level execution path to the specific evidence supporting a claim, and to see where that evidence is still incomplete.

11 Conclusion

Our paper starts from a simple problem: a successful final response alone does not explain how an LLM agent request is authenticated, authorized, routed, admitted, executed, and returned. Existing tracing, provenance, and output-level evaluation provide useful evidence, but they do not directly connect a concrete runtime outcome to the source decision and run value that selected a control-flow branch.

To address this problem, we introduce TraceClaw, a source-anchored auditing method. The method represents each Gateway stage as a verifiable evidence item containing source anchors, concrete run values, observation status, and a branch outcome. We further separate the G0–G18 Gateway control plane from the post-G18 AR0–AR6 Agent Runtime, allowing control-plane decisions and subsequent runtime behavior to be analyzed without conflating the two layers.

Across seven controlled runs, every run covers and directly observes all 19 Gateway stage outcomes, while rationale visibility remains `partial` across the Gateway stages. The runtime cases further distinguish no-tool execution, read-only retrieval, local workspace mutation, and an external side effect. The exact-prompt repeat pairs preserve the same Gateway branch structure. R3 also reveals an important evidence boundary: host-side command completion does not by itself verify a downstream external effect.

These results do not establish that OpenClaw is safe or correct in general. Instead, they show how a concrete agent execution can be translated into a set of auditable claims whose evidence strength is explicit. By distinguishing observed, source-derived, and unavailable evidence, TraceClaw provides a practical way to connect runtime traces with the source decisions they are intended to justify. This makes it possible to audit why a concrete execution path was taken and to identify where the available evidence remains incomplete.

References

[1] Benjamin H. Sigelman, Luiz Andre Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report dapper-2010-1, 2010.

[2] OpenTelemetry. Trace API Specification. https://opentelemetry.io/docs/specs/otel/trace/api/. Accessed 2026-09-30.

[3] Luc Moreau and Paolo Missier, editors. PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013.

[4] Mark Weiser. Program slicing. IEEE Transactions on Software Engineering, SE-10(4):352–357, 1984.

[5] Hiralal Agrawal and Joseph R. Horgan. Dynamic program slicing. In Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, 1990.

[6] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. Science of Computer Programming, 69(1–3):35–45, 2007.

[7] Xinru Wang, Ming Yin, Eunyee Koh, and Mustafa Doga Dogan. XAgen: An Explainability Tool for Identifying and Correcting Failures in Multi-Agent Workflows. arXiv preprint arXiv:2512.17896, 2025. https://arxiv.org/abs/2512.17896. Accessed 2026-09-24.

[8] Junpeng Wang, Yuzhong Chen, Menghai Pan, Chin-Chia Michael Yeh, and Mahashweta Das. Illuminating LLM Coding Agents: Visual Analytics for Deeper Understanding and Enhancement. arXiv preprint arXiv:2508.12555, 2025. https://arxiv.org/abs/2508.12555. Accessed 2026-09-24.

[9] Pan Hao, Dongyeop Kang, Nicholas Hinds, and Qianwen Wang. FlowForge: Guiding the Creation of Multi-agent Workflows with Interactive Visualizations as a Thinking Scaffold. IEEE VIS 2025, 2025.

[10] Fangcun Leap. Fangcun Observer: Runtime Security for AI Agents. https://fangcunleap.com/blog/observer. Accessed 2026-09-24.

[11] Vinoth Govindarajan. OpenClaw Architecture – Part 1: Control Plane, Sessions, and the Event Loop. https://theagentstack.substack.com/p/openclaw-architecture-part-1-control. Accessed 2026-09-24.

[12] Vinoth Govindarajan. OpenClaw Architecture – Part 6: Reliability, Observability, and Evaluation. https://theagentstack.substack.com/p/openclaw-architecture-part-6-reliability. Accessed 2026-09-24.

[13] Paolo Perazzo. OpenClaw Architecture, Explained: How It Works as an OS for AI Agents. https://ppaolo.substack.com/p/openclaw-system-architecture-overview. Accessed 2026-09-24.

[14] Anonymous. TraceClaw Anonymous Artifact. Anonymized artifact submitted as supplementary material.